

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт

по лабораторной работе №1
по дисциплине «**Методы оптимизации**»

Автор: Беляева Валерия Андреевна

Факультет: ФПИиКТ

Группа: К3339

Поток: 1.3

Вариант: $6 \pmod{20} = 6$

Санкт-Петербург 2026

Лабораторная работа №1. Решение задачи линейного программирования

Вариант 6. Минимизировать $Z = 2x_1 + x_2 + x_3 + 3x_4$ при условиях:

$$\begin{cases} x_1 + 2x_2 + x_4 \leq 10, \\ x_1 + x_3 + x_4 = 7, \\ x_2 + 2x_3 \geq 5, \\ x_1, x_2, x_3, x_4 \geq 0. \end{cases}$$

Работа состоит из трёх частей: решение вручную, решение в электронной таблице с помощью решателя (Solver; таблица `excel_solution.xlsx`, решение выполнено в LibreOffice Calc) и решение программой на Python. Дополнительно (бонус) решены двойственная и целочисленная задачи. Файлы работы: `excel_solution.xlsx`, `python_solution.py`; репозиторий: github.com/ValeriaBelyaeva/metopt/tree/main/lr1.

1. Решение вручную

1.1. Приведение к каноническому виду

В каноническом виде все ограничения — равенства с неотрицательными правыми частями, все переменные неотрицательны, целевая функция минимизируется. В неравенство « $\leq$ » добавляем дополнительную переменную $x_5 \geq 0$, из неравенства « $\geq$ » вычитаем $x_6 \geq 0$. Задача уже на минимум, правые части неотрицательны.

$$W(x) = 2x_1 + x_2 + x_3 + 3x_4 \rightarrow \min, \quad \begin{cases} x_1 + 2x_2 + x_4 + x_5 = 10, \\ x_1 + x_3 + x_4 = 7, \\ x_2 + 2x_3 - x_6 = 5, \\ x_1, \dots, x_6 \geq 0. \end{cases}$$

1.2. Вспомогательная задача

Для симплекс-метода нужен начальный допустимый базис. В первом уравнении есть готовая базисная переменная x_5 (входит только в него и с коэффициентом 1). Во второе и третье уравнения вводим искусственные переменные $x_7, x_8 \geq 0$ и минимизируем их сумму:

$$W'(x) = x_7 + x_8 \rightarrow \min, \quad \begin{cases} x_1 + 2x_2 + x_4 + x_5 = 10, \\ x_1 + x_3 + x_4 + x_7 = 7, \\ x_2 + 2x_3 - x_6 + x_8 = 5, \\ x_1, \dots, x_8 \geq 0. \end{cases}$$

Базис: x_5, x_7, x_8 ; свободные переменные: x_1, x_2, x_3, x_4, x_6 . Выразим W' через свободные переменные:

$$W'(x) = (7 - x_1 - x_3 - x_4) + (5 - x_2 - 2x_3 + x_6) = 12 - x_1 - x_2 - 3x_3 - x_4 + x_6.$$

Строка W' таблицы — это коэффициенты при свободных переменных (минус сумма соответствующих строк с искусственными переменными), а в правом нижнем углу стоит $-Q = -12$.

1.3. Решение вспомогательной задачи симплекс-методом

Правила пересчёта сокращённой таблицы относительно разрешающего элемента a_{rs} (как на практике):

- а) разрешающий элемент: $a'_{rs} = 1/a_{rs}$;
- б) разрешающая строка: $a'_{rj} = a_{rj}/a_{rs}$;
- в) разрешающий столбец: $a'_{is} = -a_{is}/a_{rs}$;
- г) остальные элементы («правило прямоугольника»): $a'_{ij} = a_{ij} - a_{is}a_{rj}/a_{rs}$.

Разрешающий столбец — с наименьшим отрицательным коэффициентом в строке целевой функции; разрешающая строка — с минимальным отношением b_i/a_{is} по $a_{is} > 0$. Если из базиса выходит искусственная переменная, её столбец из таблицы вычёркивается.

Таблица 0

	x_1	x_2	x_3	x_4	x_6	b
x_5	1	2	0	1	0	10
x_7	1	0	1	1	0	7
x_8	0	1	2	0	-1	5
W'	-1	-1	-3	-1	1	-12

Шаг 1. Наименьший коэффициент строки W' : -3 (столбец x_3). Отношения: $x_7: 7/1 = 7$; $x_8: 5/2 = 2,5$ — минимум в строке x_8 . Разрешающий элемент $a = 2$: x_3 входит в базис, x_8 выходит (столбец x_8 вычёркиваем).

Пример пересчёта по правилу прямоугольника: строка x_7 , столбец x_6 : $0 - \frac{1 \cdot (-1)}{2} = \frac{1}{2}$; строка W' , столбец x_2 : $-1 - \frac{(-3) \cdot 1}{2} = \frac{1}{2}$; угол: $-12 - \frac{(-3) \cdot 5}{2} = -\frac{9}{2}$.

Таблица 1

	x_1	x_2	x_4	x_6	b
x_5	1	2	1	0	10
x_7	1	$-\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{9}{2}$
x_3	0	$\frac{1}{2}$	0	$-\frac{1}{2}$	$\frac{5}{2}$
W'	-1	$\frac{1}{2}$	-1	$-\frac{1}{2}$	$-\frac{9}{2}$

Шаг 2. В строке W' два одинаковых минимальных коэффициента -1 (у x_1 и x_4); берём первый — столбец x_1 . Отношения: $x_5: 10/1 = 10$; $x_7: \frac{9}{2}/1 = 4,5$; в строке x_3 элемент 0 — не рассматриваем. Разрешающий элемент $a = 1$ в строке x_7 : x_1 входит в базис, x_7 выходит (столбец вычёркиваем).

Таблица 2

	x_2	x_4	x_6	b
x_5	$\frac{5}{2}$	0	$-\frac{1}{2}$	$\frac{11}{2}$
x_1	$-\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{9}{2}$
x_3	$\frac{1}{2}$	0	$-\frac{1}{2}$	$\frac{5}{2}$
W'	0	0	0	0

Все коэффициенты строки W' равны нулю, $\min W' = 0$, искусственных переменных в базисе не осталось. Значит, исходная задача имеет допустимые решения, и найден допустимый базис x_5, x_1, x_3 : $x_1 = \frac{9}{2}$, $x_3 = \frac{5}{2}$, $x_5 = \frac{11}{2}$, остальные равны нулю.

1.4. Переход к основной задаче

Из таблицы 2 выражаем базисные переменные через свободные ($x_{\text{баз}} = b_i - \sum_j a_{ij}x_j$):

$$x_1 = \frac{9}{2} + \frac{1}{2}x_2 - x_4 - \frac{1}{2}x_6, \quad x_3 = \frac{5}{2} - \frac{1}{2}x_2 + \frac{1}{2}x_6, \quad x_5 = \frac{11}{2} - \frac{5}{2}x_2 + \frac{1}{2}x_6.$$

Подставляем в исходную целевую функцию:

$$W(x) = 2\left(\frac{9}{2} + \frac{1}{2}x_2 - x_4 - \frac{1}{2}x_6\right) + x_2 + \left(\frac{5}{2} - \frac{1}{2}x_2 + \frac{1}{2}x_6\right) + 3x_4 = \frac{23}{2} + \frac{3}{2}x_2 + x_4 - \frac{1}{2}x_6 \rightarrow \min.$$

Строка W новой таблицы: коэффициенты $\frac{3}{2}$, 1 , $-\frac{1}{2}$, в углу $-Q = -\frac{23}{2}$.

1.5. Решение основной задачи

Таблица 3 (основная задача)

	x_2	x_4	x_6	b
x_5	$\frac{5}{2}$	0	$-\frac{1}{2}$	$\frac{11}{2}$
x_1	$-\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{9}{2}$
x_3	$\frac{1}{2}$	0	$-\frac{1}{2}$	$\frac{5}{2}$
W	$\frac{3}{2}$	1	$-\frac{1}{2}$	$-\frac{23}{2}$

Шаг 3. Отрицательный коэффициент только у x_6 ($-\frac{1}{2}$). Положительный элемент в столбце x_6 один — в строке x_1 : $\frac{9}{2} / \frac{1}{2} = 9$. Разрешающий элемент $\frac{1}{2}$: x_6 входит в базис, x_1 выходит (обычная переменная — её столбец остаётся, на месте разрешающего элемента $1 / \frac{1}{2} = 2$).

Например: строка x_5 , столбец x_2 : $\frac{5}{2} - \frac{(-\frac{1}{2})(-\frac{1}{2})}{1/2} = 2$; строка W , столбец x_4 : $1 - \frac{(-\frac{1}{2}) \cdot 1}{1/2} = 2$; угол: $-\frac{23}{2} - \frac{(-\frac{1}{2}) \cdot \frac{9}{2}}{1/2} = -7$.

Таблица 4

	x_2	x_4	x_1	b
x_5	2	1	1	10
x_6	-1	2	2	9
x_3	0	1	1	7
W	1	2	1	-7

Все коэффициенты строки W неотрицательны — найден оптимум. Базисные переменные: $x_5 = 10$, $x_6 = 9$, $x_3 = 7$; свободные $x_1 = x_2 = x_4 = 0$. В углу $-Q = -7$, значит $Z_{\min} = 7$.

1.6. Ответ и проверка

$$x^* = (0, 0, 7, 0), \quad Z(x^*) = 2 \cdot 0 + 0 + 7 + 3 \cdot 0 = 7.$$

Проверка ограничений: $0 + 0 + 0 = 0 \leq 10$ (запас $x_5 = 10$); $0 + 7 + 0 = 7$; $0 + 2 \cdot 7 = 14 \geq 5$ (излишек $x_6 = 9$).

Оптимум единственный: все оценки свободных переменных в последней таблице строго положительны (1, 2, 1). Это видно и без таблиц: из второго ограничения $x_3 = 7 - x_1 - x_4$, тогда $Z = 7 + x_1 + x_2 + 2x_4 \geq 7$, и равенство достигается только при $x_1 = x_2 = x_4 = 0$.

2. Решение в Excel-таблице (Solver, LibreOffice Calc)

На листе «ЗЛП» файла `excel_solution.xlsx` заведены изменяемые ячейки переменных $x_1..x_4$ (B4:B7), коэффициенты целевой функции (C4:C7), целевая ячейка B9 и левые части ограничений B12:B14 с формулами (правые части — в H12:H14):

Ячейка	Формула	Смысл
B9	<code>=SUMPRODUCT(\$B\$4:\$B\$7;\$C\$4:\$C\$7)</code>	$Z = 2x_1 + x_2 + x_3 + 3x_4$
B12	<code>=C12*\$B\$4+D12*\$B\$5+E12*\$B\$6+F12*\$B\$7</code>	$x_1 + 2x_2 + x_4$
B13	<code>=C13*\$B\$4+D13*\$B\$5+E13*\$B\$6+F13*\$B\$7</code>	$x_1 + x_3 + x_4$
B14	<code>=C14*\$B\$4+D14*\$B\$5+E14*\$B\$6+F14*\$B\$7</code>	$x_2 + 2x_3$

Файл открыт в LibreOffice Calc, решатель вызывается через **Сервис → Решатель...** (аналог «Поиска решения» в Excel). Задаём целевую ячейку B9 с минимизацией, изменяемые ячейки B4:B7 и три ограничения: левые части B12:B14 сравниваются с правыми частями H12:H14 со знаками $\leq$, $=$, $\geq$. В параметрах выбираем линейный решатель (симплекс-метод CoinMP) и отмечаем «Принять переменные как неотрицательные» — это условие $x_j \geq 0$.

Решатель

Целевая ячейка: solver_opt

Результат: ☐ Максимум ☒ Минимум ☐ Значение

Изменяя ячейки: \$B\$4:\$B\$7

Ограничивающие условия

Ссылка на ячейку	Операция	Значение
\$B\$12	$\leq$	\$H\$12
\$B\$13	$=$	\$H\$13
\$B\$14	$\geq$	\$H\$14

Справка Сбросить всё Параметры... Закрывать Решить

Окно «Решатель»: целевая ячейка (имя `solver_opt` из файла указывает на B9), минимум, изменяемые ячейки и ограничения

Параметры

Механизм решателя: LibreOffice CoinMP линейный решатель

Настройки:

- Ограничение поиска решения по времени (секунд): 100
- ☒ Ограничить глубину ветвей и границ
- ☒ Принять переменные как неотрицательные
- ☐ Принять переменные как целочисленные
- Уровень эпсилон (0-3): 0

Правка... Справка OK Отменить

Результат

Процесс решения успешно завершён.

Результат: 7

Сохранить результат или восстановить предыдущие значения?

Сохранить результат Восстановить предыдущие

Параметры: линейный решатель CoinMP, переменные неотрицательные

Результат: решение найдено, целевая функция = 7

Решатель сообщает, что процесс решения успешно завершён, результат — 7. В ячейках B4:B7 получаем $x^* = (0, 0, 7, 0)$, в B9 — $Z = 7$; левые части ограничений: $0 \leq 10$, $7 = 7$, $14 \geq 5$. Результат совпадает с ручным решением.

На листах «Двойственная» и «Целочисленная» того же файла заведены модели для бонусных задач (раздел 4).

3. Решение программой (Python)

Программа реализует тот же алгоритм, что и ручное решение, без библиотек с готовым симплекс-методом (не используются ни `scipy.linprog`, ни `numpy`: вычисления ведутся в обыкновенных дробях `fractions.Fraction`, поэтому промежуточные таблицы получаются точно такими же, как при решении вручную). Код универсальный: задача задаётся вектором c , списком ограничений вида (коэффициенты, знак, правая часть) и направлением оптимизации; поддерживаются $\leq$, $=$, $\geq$, задачи на минимум и максимум, переменные любого знака. Каждый шаг решения — отдельная функция:

Функция	Шаг
<code>to_canonical</code>	приведение к каноническому виду (балансовые переменные, $b \geq 0$, $\max \rightarrow \min$, расщепление свободных переменных)
<code>build_auxiliary</code>	построение вспомогательной задачи: поиск готовых базисных столбцов, ввод искусственных переменных, строка W'
<code>choose_pivot</code>	выбор разрешающего элемента; обнаружение неограниченности целевой функции
<code>jordan_step</code>	пересчёт сокращённой таблицы по правилам а)–г), вычёркивание столбца искусственной переменной
<code>run_simplex</code>	итерации симплекс-метода до оптимума
<code>to_main_problem</code>	переход к основной задаче: проверка $\min W' = 0$ (иначе решений нет), вывод искусственных переменных из базиса, построение строки W
<code>extract_answer, solve</code>	ответ x^* , $Z(x^*)$; полный цикл решения
<code>build_dual, branch_and_bound</code>	бонус: двойственная задача и целочисленная задача (метод ветвей и границ)

Для проверки корректности программа была прогнана на 600 случайных задачах (2–5 переменных, 1–4 ограничения разных типов, $\min$ и $\max$) и сравнена с `scipy.optimize.linprog`: ответы совпали в 599 случаях, включая «решений нет» (пустая область и неограниченная функция); в оставшемся случае ошибся `scipy` — назвал задачу недопустимой, хотя у неё есть допустимая точка, а целевая функция не ограничена (это и выдала программа). `scipy` использовался только в проверке и в решение не входит.

Листинг `python_solution.py`

```
"""
ЛР1. Решение задачи линейного программирования симплекс-методом.
Беляева Валерия Андреевна, КЗ339, поток 1.3, вариант 6.

Метод — как на практике: сокращённая симплекс-таблица
(строки — базисные переменные, столбцы — свободные переменные и b,
последняя строка — коэффициенты целевой функции и -Q в углу).
Пересчёт таблицы — по правилам а)–г) (метод Жордана-Гаусса).
Вычисления ведутся в обыкновенных дробях (fractions.Fraction),
поэтому промежуточные таблицы совпадают с ручным решением.

Готовые реализации симплекс-метода (scipy.linprog, pulp и т.п.) не используются.
"""
from fractions import Fraction
from math import floor, ceil

LE, EQ, GE = "<=", "=", ">="

# ----- вывод таблиц

def fmt(v):
    v = Fraction(v)
    return str(v.numerator) if v.denominator == 1 else f"{v.numerator}/{v.denominator}"

def print_table(t, title=""):
    """Печать сокращённой симплекс-таблицы."""
    if title:
        print(title)
    w = 7
    print(" " * 6 + "".join(f"{n:>{w}}" for n in t["free"]) + f"{'b':>{w}}")
    for name, row in zip(t["basis"], t["rows"]):
        print(f"{name:>5} " + "".join(f"{fmt(v):>{w}}" for v in row))
    print(f"{t['obj_name']:>5} " + "".join(f"{fmt(v):>{w}}" for v in t["obj"]))
```

```

print()

# ----- шаг 1: канонический вид

def to_canonical(c, constraints, sense="min", free_vars=()):
    """Приведение ЗЛП к каноническому виду.

    c          — коэффициенты целевой функции;
    constraints — список (коэффициенты, знак, правая часть), знак: '<=', '=', '>=';
    sense      — 'min' или 'max' (max f сводится к min (-f));
    free_vars  — индексы переменных без ограничения на знак (x = x' - x'').

    Возвращает словарь: матрица A, вектор b >= 0, вектор c (на минимум),
    имена переменных и способ восстановить исходные переменные.
    """
    n = len(c)
    sign = 1 if sense == "min" else -1
    names, cost, columns, recover = [], [], [], [] # recover[k] = [(номер столбца, множитель)]

    # исходные переменные (свободные расщепляем на разность двух неотрицательных)
    for k in range(n):
        col = [Fraction(a[k]) for a, _, _ in constraints]
        if k in free_vars:
            names += [f"x{k + 1}'", f"x{k + 1}''"]
            cost += [sign * Fraction(c[k]), -sign * Fraction(c[k])]
            columns += [col, [-v for v in col]]
            recover.append([(len(names) - 2, 1), (len(names) - 1, -1)])
        else:
            names.append(f"x{k + 1}")
            cost.append(sign * Fraction(c[k]))
            columns.append(col)
            recover.append([(len(names) - 1, 1)])

    # дополнительные (балансовые) переменные: +s для '<=', -s для '>='
    m = len(constraints)
    extra = n + 1
    for i, (_, rel, _) in enumerate(constraints):
        if rel == EQ:
            continue
        col = [Fraction(0)] * m
        col[i] = Fraction(1 if rel == LE else -1)
        names.append(f"x{extra}")
        extra += 1
        cost.append(Fraction(0))
        columns.append(col)

    A = [[columns[j][i] for j in range(len(columns))] for i in range(m)]
    b = [Fraction(rhs) for _, _, rhs in constraints]

    # правые части должны быть неотрицательными
    for i in range(m):
        if b[i] < 0:
            A[i] = [-v for v in A[i]]
            b[i] = -b[i]

    return {"A": A, "b": b, "c": cost, "names": names,
            "recover": recover, "sign": sign, "n_orig": n}

def print_canonical(canon):
    A, b, names = canon["A"], canon["b"], canon["names"]

    def lin(coefs):
        parts = []
        for a, nm in zip(coefs, names):
            if a == 0:
                continue
            s = "-" if a < 0 else "+"
            mag = "" if abs(a) == 1 else fmt(abs(a))
            parts.append(f"{s} {mag} {nm}")
        text = " ".join(parts) or "0"
        return text[2:] if text.startswith("+ ") else text

    print("Канонический вид:")
    print(f" W(x) = {lin(canon['c'])} -> min")
    for row, rhs in zip(A, b):
        print(f" {lin(row)} = {fmt(rhs)}")
    print(f" {'', '.join(names)} >= 0\n")

# ----- шаг 2: вспомогательная задача

def build_auxiliary(canon):
    """Построение вспомогательной задачи.

    В строке, где уже есть «готовая» базисная переменная (единичный столбец),
    она и становится базисной; в остальные строки вводятся искусственные
    переменные. W'(x) = сумма искусственных переменных -> min.
    """

```

```

A, b, names = canon["A"], canon["b"], canon["names"]
m, n = len(A), len(names)

basis = [None] * m
for j in range(n):
    col = [A[i][j] for i in range(m)]
    nonzero = [i for i in range(m) if col[i] != 0]
    if len(nonzero) == 1 and col[nonzero[0]] == 1 and basis[nonzero[0]] is None:
        basis[nonzero[0]] = j

art_names = []
next_index = n + 1
for i in range(m):
    if basis[i] is None:
        art_names.append(f"x_{next_index}")
        basis[i] = art_names[-1]
        next_index += 1
    else:
        basis[i] = names[basis[i]]

free = [nm for nm in names if nm not in basis]
col_of = {nm: j for j, nm in enumerate(names)}
rows = [[A[i][col_of[nm]] for nm in free] + [b[i]] for i in range(m)]

# W' = сумма искусственных = сумма (b_i - sum a_ij x_j) по их строкам
obj = [Fraction(0)] * (len(free) + 1)
for i in range(m):
    if basis[i] in art_names:
        for j in range(len(free) + 1):
            obj[j] -= rows[i][j]          # в углу таблицы стоит -Q

return {"basis": basis, "free": free, "rows": rows, "obj": obj,
        "obj_name": "W'", "artificial": set(art_names)}

# ----- шаг 3: пересчёт таблицы

def choose_pivot(t):
    """Выбор разрешающего элемента.

    Столбец — наименьший отрицательный коэффициент целевой строки
    (если таких нет — оптимум найден, возвращаем None).
    Строка — минимальное отношение b_i / a_is по a_is > 0
    (если таких нет — целевая функция не ограничена снизу).
    """
    d = t["obj"][-1]
    s = min(range(len(d)), key=lambda j: (d[j], j)) if d else None
    if s is None or d[s] >= 0:
        return None

    ratios = [(row[-1] / row[s], i) for i, row in enumerate(t["rows"]) if row[s] > 0]
    if not ratios:
        raise ValueError("Целевая функция не ограничена снизу — решений нет")
    _, r = min(ratios)
    return r, s

def jordan_step(t, r, s):
    """Пересчёт сокращённой таблицы относительно элемента a_rs:
    а) a_rs' = 1 / a_rs;
    б) строка: a_rj' = a_rj / a_rs;
    в) столбец: a_is' = -a_is / a_rs;
    г) остальные: a_ij' = a_ij - a_is * a_rj / a_rs.
    Базисная переменная строки r и свободная столбца s меняются местами.
    Если из базиса вышла искусственная переменная — её столбец удаляется.
    """
    rows = t["rows"] + [t["obj"]]
    p = rows[r][s]
    new = [row[:] for row in rows]
    for i in range(len(rows)):
        for j in range(len(rows[0])):
            if i == r and j == s:
                new[i][j] = 1 / p
            elif i == r:
                new[i][j] = rows[r][j] / p
            elif j == s:
                new[i][j] = -rows[i][s] / p
            else:
                new[i][j] = rows[i][j] - rows[i][s] * rows[r][j] / p

    t["basis"][r], t["free"][s] = t["free"][s], t["basis"][r]
    t["rows"], t["obj"] = new[:-1], new[-1]

    if t["free"][s] in t.get("artificial", ()):
        drop_column(t, s)

def drop_column(t, s):
    del t["free"][s]
    for row in t["rows"]:

```

```

del row[s]
del t["obj"][s]

def run_simplex(t, verbose=True, start=0):
    """Итерации симплекс-метода до оптимума. Возвращает номер последней таблицы."""
    k = start
    while True:
        pivot = choose_pivot(t)
        if pivot is None:
            return k
        r, s = pivot
        if verbose:
            print(f"Разрешающий элемент: строка {t['basis'][r]}, столбец {t['free'][s]}, "
                  f"a = {fmt(t['rows'][r][s])} ({t['free'][s]} входит в базис, "
                  f"{t['basis'][r]} выходит)")
        jordan_step(t, r, s)
        k += 1
        if verbose:
            print_table(t, f"Таблица {k}:")

# ----- шаг 4: переход к основной задаче

def to_main_problem(t, canon, verbose=True):
    """Переход от вспомогательной задачи к основной.

    1) Проверяем, что  $\min W' = 0$ , иначе допустимых решений нет.
    2) Если искусственная переменная осталась в базисе (на нулевом уровне),
       выводим её, либо удаляем строку, если она линейно зависима.
    3) Строим строку целевой функции:  $Z$  выражается через свободные переменные.
    """
    if t["obj"][-1] != 0:
        raise ValueError("min W' > 0: область допустимых решений пуста – решений нет")

    i = 0
    while i < len(t["basis"]):
        if t["basis"][i] in t["artificial"]:
            cols = [j for j, v in enumerate(t["rows"][i][:-1]) if v != 0]
            if cols:
                jordan_step(t, i, cols[0])
            else:
                del t["basis"][i]
                del t["rows"][i]
                continue
        i += 1
    for nm in [nm for nm in t["free"] if nm in t["artificial"]]:
        drop_column(t, t["free"].index(nm))

    cost = dict(zip(canon["names"], canon["c"]))
    #  $Z = Q + \sum d_j x_j$ , где  $x_{\text{базисная}} = b_i - \sum a_{ij} x_j$ 
    obj = []
    for j, nm in enumerate(t["free"]):
        obj.append(cost[nm] - sum(cost[bn] * row[j] for bn, row in zip(t["basis"], t["rows"])))
    Q = sum(cost[bn] * row[-1] for bn, row in zip(t["basis"], t["rows"]))
    t["obj"] = obj + [-Q]
    t["obj_name"] = "W"
    t["artificial"] = set()
    if verbose:
        print("Переход к основной задаче:  $W(x) =$ 
              + f"{fmt(Q)} " + " ".join(f"{'+' if d >= 0 else '-'} "
              f"{' ' if abs(d) == 1 else fmt(abs(d))}{nm}"
              for d, nm in zip(obj, t["free"])) + " -> min")
    return t

# ----- шаг 5: ответ

def extract_answer(t, canon):
    values = {nm: Fraction(0) for nm in canon["names"]}
    for nm, row in zip(t["basis"], t["rows"]):
        if nm in values:
            values[nm] = row[-1]
    names = canon["names"]
    x = [sum(mult * values[names[col]] for col, mult in parts) for parts in canon["recover"]]
    f = canon["sign"] * (-t["obj"][-1]) # Q = -(угловой элемент)
    return x, f

def solve(c, constraints, sense="min", free_vars=(), verbose=True):
    """Полное решение ЗЛП: канонический вид -> вспомогательная задача ->
    решение -> переход к основной -> решение -> ответ."""
    canon = to_canonical(c, constraints, sense, free_vars)
    if verbose:
        print_canonical(canon)

    t = build_auxiliary(canon)
    if verbose:
        art = " ".join(sorted(t["artificial"])) or "не нужны"
        print(f"Вспомогательная задача: искусственные переменные – {art}; ")

```

```

        f"W'(x) = сумма искусственных -> min")
    print_table(t, "Таблица 0:")
    k = run_simplex(t, verbose)

    to_main_problem(t, canon, verbose)
    if verbose:
        print_table(t, f"Таблица {k + 1} (основная задача):")
    run_simplex(t, verbose, start=k + 1)

    x, f = extract_answer(t, canon)
    if verbose:
        print("Все коэффициенты целевой строки неотрицательны – оптимум найден.")
    return x, f

# ----- бонус: двойственная задача

def build_dual(c, constraints):
    """Двойственная задача к min c^T x, x >= 0.
    Ограничения '<=' предварительно умножаются на -1 (приводим к '>='),
    тогда: y_i >= 0 для '>=', y_i – любого знака для '='.
    Двойственная: max b^T y, A^T y <= c."""
    rows = []
    for a, rel, rhs in constraints:
        if rel == LE:
            rows.append([-v for v in a], GE, -rhs)
        else:
            rows.append(list(a), rel, rhs)
    b = [rhs for _, _, rhs in rows]
    dual_constraints = [[([rows[i][0][k] for i in range(len(rows))], LE, c[k])
                        for k in range(len(c)))]
    free = tuple(i for i, (_, rel, _) in enumerate(rows) if rel == EQ)
    return b, dual_constraints, free

# ----- бонус: целочисленная задача (ветви и границы)

def branch_and_bound(c, constraints, sense="min", best=None, depth=0, max_depth=40):
    """Метод ветвей и границ поверх собственного симплекс-метода.
    max_depth ограничивает ветвление, если допустимая область неограничена."""
    if depth > max_depth:
        print(" " * depth + "достигнута предельная глубина ветвления – ветвь отброшена")
        return best
    try:
        x, f = solve(c, constraints, sense, verbose=False)
    except ValueError:
        return best
    better = (lambda a, b: a < b) if sense == "min" else (lambda a, b: a > b)
    if best is not None and not better(f, best[1]):
        return best
    frac = [k for k, v in enumerate(x) if v.denominator != 1]
    pad = " " * depth
    if not frac:
        print(f"{pad}целочисленное решение x = {[fmt(v) for v in x]}, f = {fmt(f)}")
        return (x, f)
    k = frac[0]
    print(f"{pad}x{k + 1} = {fmt(x[k])} нецелое -> ветвим: "
          f"x{k + 1} <= {floor(x[k])} | x{k + 1} >= {ceil(x[k])}")
    unit = [1 if j == k else 0 for j in range(len(c))]
    best = branch_and_bound(c, constraints + [(unit, LE, floor(x[k]))], sense, best,
                           depth + 1, max_depth)
    best = branch_and_bound(c, constraints + [(unit, GE, ceil(x[k]))], sense, best,
                           depth + 1, max_depth)
    return best

# ----- вариант 6

if __name__ == "__main__":
    # Минимизировать Z = 2x1 + x2 + x3 + 3x4
    c = [2, 1, 1, 3]
    constraints = [
        ([1, 2, 0, 1], LE, 10), # x1 + 2x2 + x4 <= 10
        ([1, 0, 1, 1], EQ, 7), # x1 + x3 + x4 = 7
        ([0, 1, 2, 0], GE, 5), # x2 + 2x3 >= 5
    ]

    print("=" * 60 + "\nПрямая задача (вариант 6)\n" + "=" * 60)
    x, z = solve(c, constraints, "min")
    print(f"\nОптимальная точка: x* = ({', '.join(fmt(v) for v in x)})")
    print(f"Значение целевой функции: Z(x*) = {fmt(z)}")

    print("\n" + "=" * 60 + "\nБонус 1: двойственная задача\n" + "=" * 60)
    b_dual, dual_constraints, free = build_dual(c, constraints)
    print("max " + " " + ".join(f"({v})y{i + 1}" for i, v in enumerate(b_dual)))
    for a, rel, rhs in dual_constraints:
        print(" " + " " + ".join(f"({v})y{i + 1}" for i, v in enumerate(a)) + f" {rel} {rhs}")
    print(f" свободные по знаку: {'', '.join(f'y{i + 1}' for i in free)}; остальные >= 0\n")
    y, w = solve(b_dual, dual_constraints, "max", free_vars=free, verbose=False)
    print(f"y* = ({', '.join(fmt(v) for v in y)}), max = {fmt(w)}")

```

```

print(f"Z(x*) = {fmt(z)} = W(y*) = {fmt(w)} — теорема двойственности выполняется")

print("\n" + "=" * 60 + "\nБонус 2: целочисленная задача (x — целые)\n" + "=" * 60)
x1, z1 = branch_and_bound(c, constraints, "min")
print(f"x* = ({', '.join(fmt(v) for v in xi)}), Z = {fmt(z1)}")

```

Результат работы программы

```

=====
Прямая задача (вариант 6)
=====
Канонический вид:
W(x) = 2x1 + x2 + x3 + 3x4 -> min
x1 + 2x2 + x4 + x5 = 10
x1 + x3 + x4 = 7
x2 + 2x3 - x6 = 5
x1, x2, x3, x4, x5, x6 >= 0

Вспомогательная задача: искусственные переменные — x7, x8; W'(x) = сумма искусственных -> min
Таблица 0:

```

	x1	x2	x3	x4	x6	b
x5	1	2	0	1	0	10
x7	1	0	1	1	0	7
x8	0	1	2	0	-1	5
W'	-1	-1	-3	-1	1	-12

```

Разрешающий элемент: строка x8, столбец x3, a = 2 (x3 входит в базис, x8 выходит)
Таблица 1:

```

	x1	x2	x4	x6	b
x5	1	2	1	0	10
x7	1	-1/2	1	1/2	9/2
x3	0	1/2	0	-1/2	5/2
W'	-1	1/2	-1	-1/2	-9/2

```

Разрешающий элемент: строка x7, столбец x1, a = 1 (x1 входит в базис, x7 выходит)
Таблица 2:

```

	x2	x4	x6	b
x5	5/2	0	-1/2	11/2
x1	-1/2	1	1/2	9/2
x3	1/2	0	-1/2	5/2
W'	0	0	0	0

```

Переход к основной задаче: W(x) = 23/2 + 3/2x2 + x4 - 1/2x6 -> min
Таблица 3 (основная задача):

```

	x2	x4	x6	b
x5	5/2	0	-1/2	11/2
x1	-1/2	1	1/2	9/2
x3	1/2	0	-1/2	5/2
W	3/2	1	-1/2	-23/2

```

Разрешающий элемент: строка x1, столбец x6, a = 1/2 (x6 входит в базис, x1 выходит)
Таблица 4:

```

	x2	x4	x1	b
x5	2	1	1	10
x6	-1	2	2	9
x3	0	1	1	7
W	1	2	1	-7

```

Все коэффициенты целевой строки неотрицательны — оптимум найден.

Оптимальная точка: x* = (0, 0, 7, 0)
Значение целевой функции: Z(x*) = 7

=====
Бонус 1: двойственная задача
=====
max (-10)y1 + (7)y2 + (5)y3
(-1)y1 + (1)y2 + (0)y3 <= 2
(-2)y1 + (0)y2 + (1)y3 <= 1
(0)y1 + (1)y2 + (2)y3 <= 1
(-1)y1 + (1)y2 + (0)y3 <= 3
свободные по знаку: y2; остальные >= 0

y* = (0, 1, 0), max = 7
Z(x*) = 7 = W(y*) = 7 — теорема двойственности выполняется

=====
Бонус 2: целочисленная задача (x — целые)
=====
целочисленное решение x = ['0', '0', '7', '0'], f = 7
x* = (0, 0, 7, 0), Z = 7

```

Таблицы, которые печатает программа, совпадают с таблицами 0–4 ручного решения, ответ — $x^* = (0, 0, 7, 0)$, $Z = 7$ — совпадает с ручным решением и с Excel.

4. Бонус: двойственная и целочисленная задачи

4.1. Двойственная задача

Приведём прямую задачу на минимум к виду, где все неравенства « $\geq$ »: первое ограничение умножим на -1 : $-x_1 - 2x_2 - x_4 \geq -10$. Каждому ограничению соответствует двойственная переменная: $y_1 \geq 0$ (неравенство), y_2 — любого знака (равенство), $y_3 \geq 0$ (неравенство). Каждой переменной $x_j \geq 0$ — ограничение « $\leq c_j$ »:

$$\Phi(y) = -10y_1 + 7y_2 + 5y_3 \rightarrow \max, \quad \begin{cases} -y_1 + y_2 \leq 2, \\ -2y_1 + y_3 \leq 1, \\ y_2 + 2y_3 \leq 1, \\ -y_1 + y_2 \leq 3, \\ y_1, y_3 \geq 0, y_2 \in \mathbb{R}. \end{cases}$$

Решение через условия дополняющей нежёсткости: в оптимуме прямой задачи первое ограничение выполнено строго ($x_5 = 10 > 0$) $\Rightarrow y_1 = 0$; третье тоже строго ($x_6 = 9 > 0$) $\Rightarrow y_3 = 0$; $x_3 = 7 > 0 \Rightarrow$ третье двойственное ограничение — равенство: $y_2 + 2y_3 = 1 \Rightarrow y_2 = 1$. Проверка остальных: $1 \leq 2$, $0 \leq 1$, $1 \leq 3$ — выполнены.

$y^* = (0, 1, 0)$, $\Phi(y^*) = 7 = Z(x^*)$ — значения прямой и двойственной задач совпадают.

Двойственные оценки показывают, как меняется оптимум при изменении правых частей: увеличение правой части равенства $x_1 + x_3 + x_4 = 7$ на единицу увеличивает $Z_{\min}$ на $y_2 = 1$, а первое и третье ограничения не активны, поэтому их оценки нулевые. Эту же задачу решает программа (функция `build_dual`, свободная y_2 расщепляется на $y'_2 - y''_2$) и Excel (лист «Двойственная»: флажок неотрицательности снят, $y_1, y_3 \geq 0$ заданы явно) — ответ тот же.

4.2. Целочисленная задача

Если потребовать $x_j \in \mathbb{Z}$, ответ не меняется: оптимум непрерывной задачи $x^* = (0, 0, 7, 0)$ уже целочисленный, поэтому метод ветвей и границ (функция `branch_and_bound`) завершается в корне дерева без ветвления, а в Excel (лист «Целочисленная», ограничение B4:B7 = *цел*) получается то же $Z = 7$. Ветвление в программе проверено на учебном примере $5x_1 + 4x_2 \rightarrow \max$, $6x_1 + 4x_2 \leq 24$, $x_1 + 2x_2 \leq 6$: непрерывный оптимум $(3, 1, 5)$, целочисленный — $(4, 0)$ со значением 20.

5. Рефлексивный вывод

В работе одна и та же задача решена тремя способами — вручную, в Excel и программой, и все три дали один ответ $x^* = (0, 0, 7, 0)$, $Z = 7$. Самым полезным оказалось ручное решение: только пересчитывая таблицы по правилам а)–г) своими руками, понимаешь, зачем нужна вспомогательная задача и почему из её оптимума можно перейти к основной. Труднее всего было не запутаться в знаках при пересчёте (особенно в разрешающем столбце, где знак меняется) и в том, какие столбцы вычёркиваются, а какие остаются при смене базиса; помогала проверка — подстановка найденной точки в ограничения и сравнение с Excel. Excel решает задачу за секунды, но ничего не объясняет — это удобный способ проверить ответ. Программа потребовала аккуратно разделить алгоритм на шаги; чтобы её таблицы можно было сверять с ручными, я считала в обыкновенных дробях, а для уверенности в корректности сравнила её с готовым решателем на сотнях случайных задач. Отдельно было интересно увидеть двойственную задачу «в цифрах»: её оптимум совпал с прямым, а двойственные переменные показали, какое ограничение на самом деле определяет ответ.